

JARFIN

**Alessandro
Biscaro**

Mat. 1087892

**Davide
Bonsembiante**

Mat. 1086863

**Alessandro
Rocco**

Mat. 1081179



Panoramica del Progetto



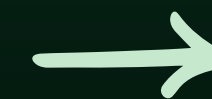
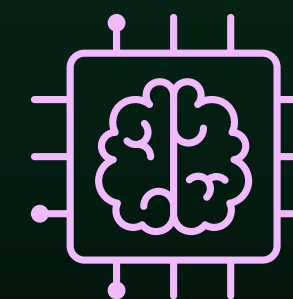
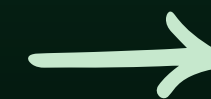
Problema iniziale:

- **Scarsa modularità**
- **Sistemi finanziari monolitici**
- **Nessun supporto NLU**
- **Inserimento dati manuale**

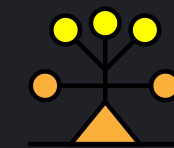


Obiettivo:

- **Architettura a microservizi**
- **Sistema scalabile e modulare**
- **Analisi automatica dati**
- **Interazione tramite NLU**



Obiettivo



Architettura a microservizi



Comunicazione tramite API REST



Separazione delle responsabilità



Sistema estendibile

Difficoltà Riscontrate

*Quali difficoltà abbiamo incontrato
durante il progetto?*



**Progettazione
architettura
microservizi**



**Definizione
comunicazione
REST**



**Implementazione
modulo NLU**



**Planning in
sessione**

Paradigma di Programmazione

01. Core & Backend

- *Java 21*
- *Spring Boot 3.2*
- *Spring Cloud Gateway*
- *Hibernate*

02. Frontend & Interazione

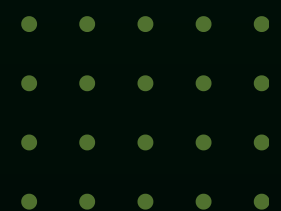
- *Thymeleaf*
- *Bootstrap 5*
- *Web Speech API*

03. Persistenza & Build

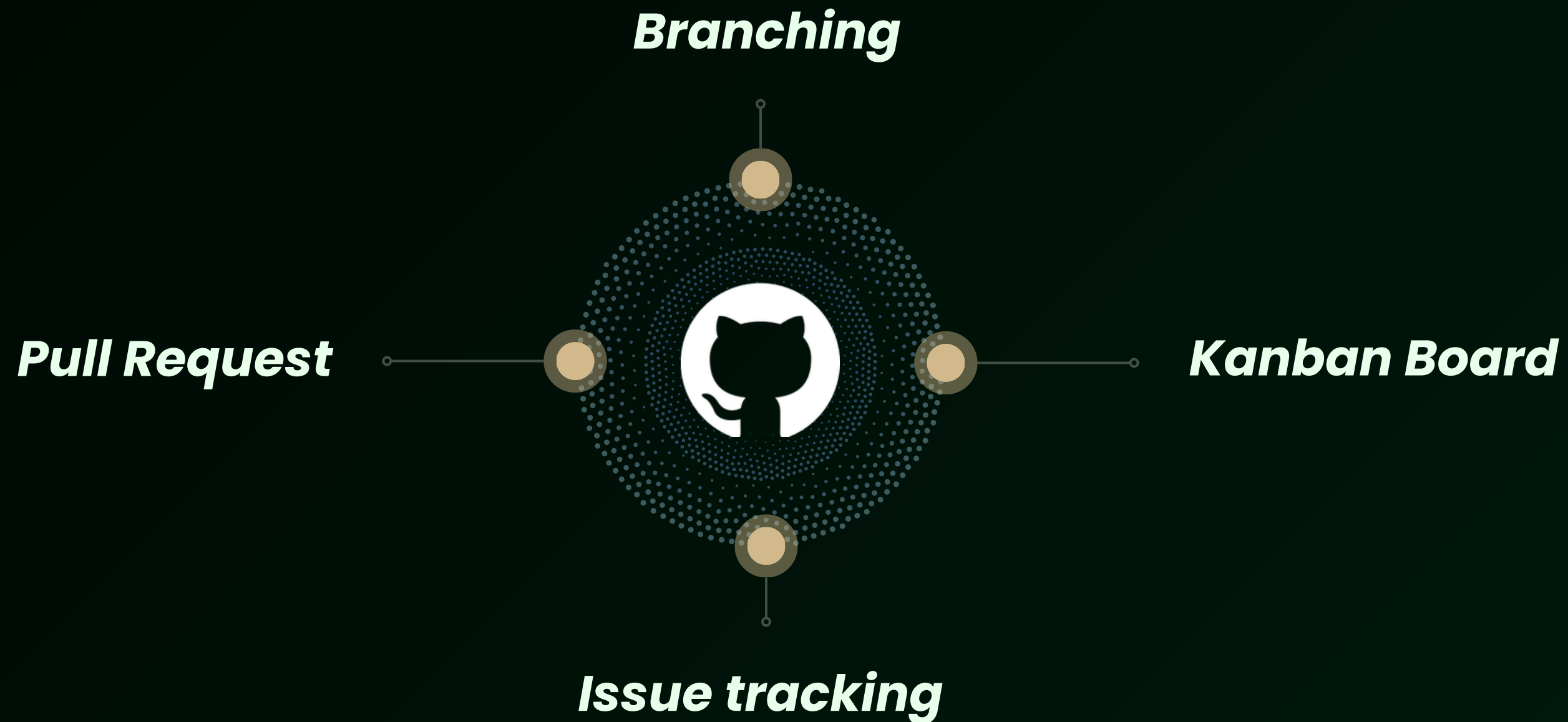
- *Spring Data JPA*
- *PostgreSQL (Docker)*
- *Maven*
- *Lombok*

04. Testing & Qualità

- *JUnit 5*
- *Mockito*
- *SonarLint*
- *CodeMR*
- *Postman*
- *EclEmma*



Software Configuration Management



Software lifecycle

01



Iterazione 0

*Analisi requisiti, setup
ambiente e modellazione
UML completa.*

02



Iterazione 1

*Microservizio Accounting,
persistenza database e API
REST CRUD.*

03



Iterazione 2

*Microservizio Analytics,
aggregazione dati e calcolo
statistiche.*

04



Iterazione 3

*Motore NLU, configurazione API
Gateway e interfaccia Web
vocale.*

Requisiti Funzionali



CRUD Transazioni

Gestione completa del ciclo di vita dei dati (creazione, lettura, modifica, eliminazione) per tracciare entrate e uscite in modo persistente.



Report Finanziari

Calcolo in tempo reale dei saldi aggregati e generazione di statistiche per categoria tramite il microservizio di Analytics.



Comandi Vocali

Interpretazione ed esecuzione di comandi in linguaggio naturale per un inserimento dati rapido e "hands-free" tramite interfaccia Web.

Requisiti Non Funzionali



Bassa latenza

Elaborazione istantanea del testo grazie a un algoritmo NLU deterministico $O(n)$, evitando i tempi di risposta delle API AI esterne.



Scalabilità servizi

Architettura distribuita che permette di scalare o replicare singolarmente i microservizi (es. Accounting) in base al carico di lavoro



Modularità componenti

Netta separazione delle responsabilità con comunicazione via REST e Gateway, facilitando la manutenzione e l'aggiornamento indipendente dei moduli.

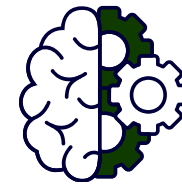


UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Architettura

Il sistema usa **microservizi distribuiti** con gestione dati autonoma.

L'**API Gateway** funge da Single Entry Point, centralizzando routing e sicurezza per disaccoppiare il frontend dal backend.

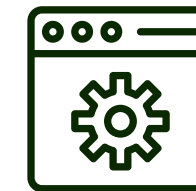


Architettura

Stile a Microservizi distribuiti con pattern MVC interno ai servizi

Separazione

Disaccoppiamento netto tra Logica - Interfaccia - Routing



API Gateway

come unico punto di ingresso (**Single Entry Point**)



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Microservizi

```
AccountingServiceApplication.java
org.springframework.boot.SpringApplication;

AccountingServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(AccountingServiceApplication.class, args);
    }
}

AccountingServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(AccountingServiceApplication.class, args);
    }
}
```

ACCOUNTING SERVICE

```
AnalyticsServiceApplication.java
org.springframework.boot.SpringApplication;

AnalyticsServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(AnalyticsServiceApplication.class, args);
    }
}
```

ANALYTICS SERVICE

```
NLUServiceApplication.java
org.springframework.boot.SpringApplication;

NLUServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(NLUServiceApplication.class, args);
    }
}
```

NLU SERVICE

```
APIGatewayApplication.java
org.springframework.boot.SpringApplication;

APIGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(APIGatewayApplication.class, args);
    }
}
```

API GATEWAY



Analytics Engine

01. Tecnica:

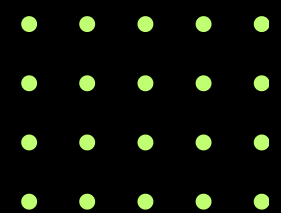
*Aggregazione statistica dei dati
tramite approccio **Single-Pass***

02. Complessità:

***$O(N)$** sul numero totale delle
transazioni registrate*

03. Vantaggio:

*Consente di generare report, saldi
e KPI finanziari in tempo reale*





NLU Engine

01. Tecnica:

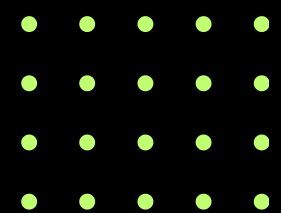
*Pattern Matching Deterministico
basato su Regex ottimizzate.*

02. Complessità:

*$O(N)$ sulla lunghezza della
stringa in input*

03. Vantaggio:

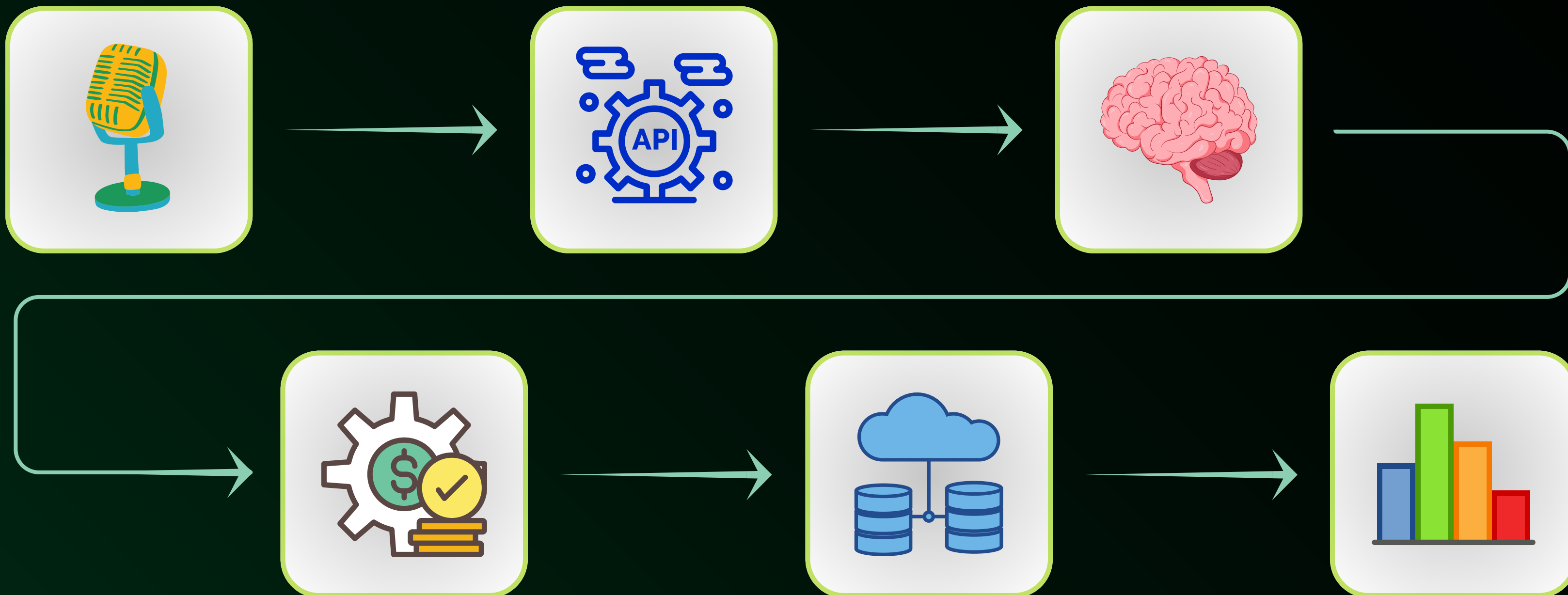
*Elaborazione locale (**Edge AI**)
a latenza zero.*





UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Flusso Operativo





Attuale

Cosa abbiamo usato per implementare?

- **Java 21 & Spring Boot 3:** sviluppo backend
- **Thymeleaf & Web Speech API:** frontend server-side e interfaccia vocale nativa

Quali parti abbiamo implementato?

- **Microservizi Core:** Accounting, Analytics
- **Motore NLU:** Algoritmo di parsing per tradurre la voce in comandi JSON
- **API Gateway:** Orchestratore per il routing e la sicurezza.

Cosa abbiamo usato per fare testing?

- **JUnit 5 & Mockito:** Test isolati e mocking delle dipendenze
- **SonarLint:** Identificazione e correzione di code smells.
- **CodeMR:** Visualizzazione delle metriche software
- **Postman:** Validazione dei flussi End-to-End sulle API REST



Futuro

Quali sono i prossimi sviluppi previsti?

- **Categorizzazione via Machine Learning:** Sostituire l'attuale logica a Regex con un modello NLP addestrato per predire automaticamente la categoria di spesa basandosi sullo storico dell'utente.
- **Autenticazione Centralizzata (OAuth2):** Implementare un sistema di Single Sign-On (SSO) basato sullo standard OAuth2/OpenID Connect (utilizzando ad esempio Keycloak o identity provider esterni come Google)
- **Deployment Cloud Native:** Dockerizzare tutti i microservizi e orchestrarli tramite Kubernetes per garantire scalabilità automatica e alta disponibilità su infrastruttura Cloud.
- **Integrazione OCR per Scontrini:** Implementare una funzionalità di scansione ottica che permetta di compilare automaticamente una transazione semplicemente fotografando lo scontrino cartaceo.

Ora è arrivato il momento di illustrare JarFin

*Preparati a scoprire come **JarFin** trasforma la tua **voce** in **dati finanziari** grazie a un'architettura potente e modulare*
